

DBSCAN

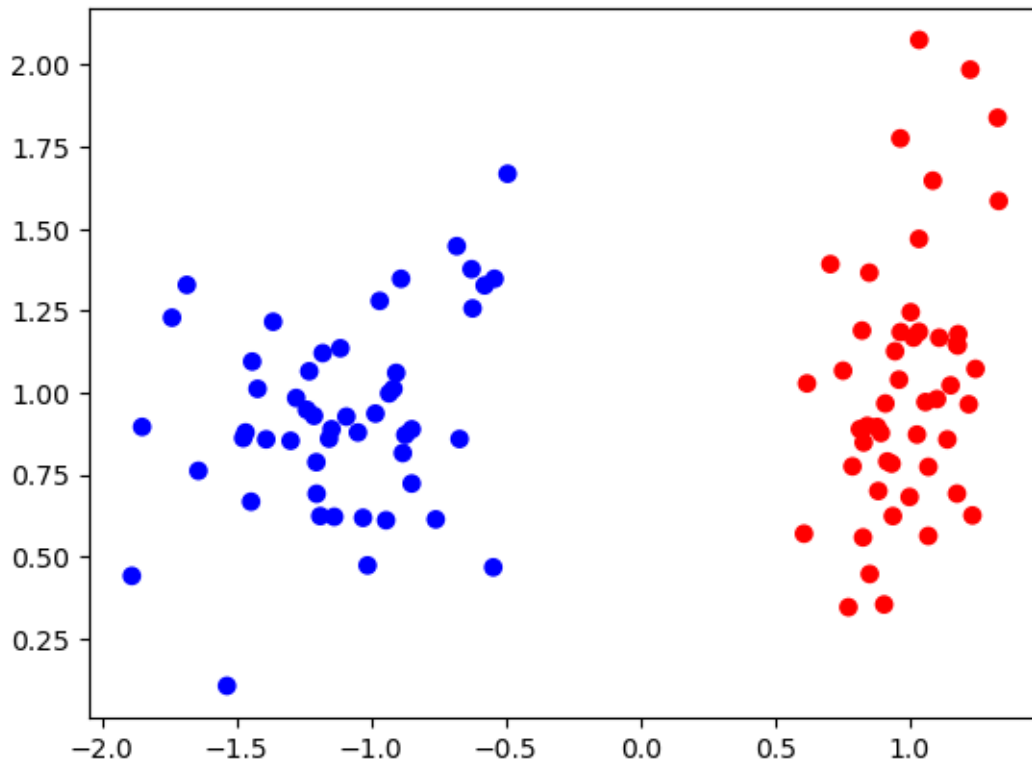
Guillermo Ruiz

Se muestran ejemplos del uso de DBSCAN. Las funciones `make_moons`, `make_circles`, `make_classification`, y `make_s_curve` crean conjuntos de puntos con diferentes formas.

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import DBSCAN
from sklearn.datasets import make_blobs
from sklearn.datasets import make_moons, make_circles, make_classification, make_s_curve
```

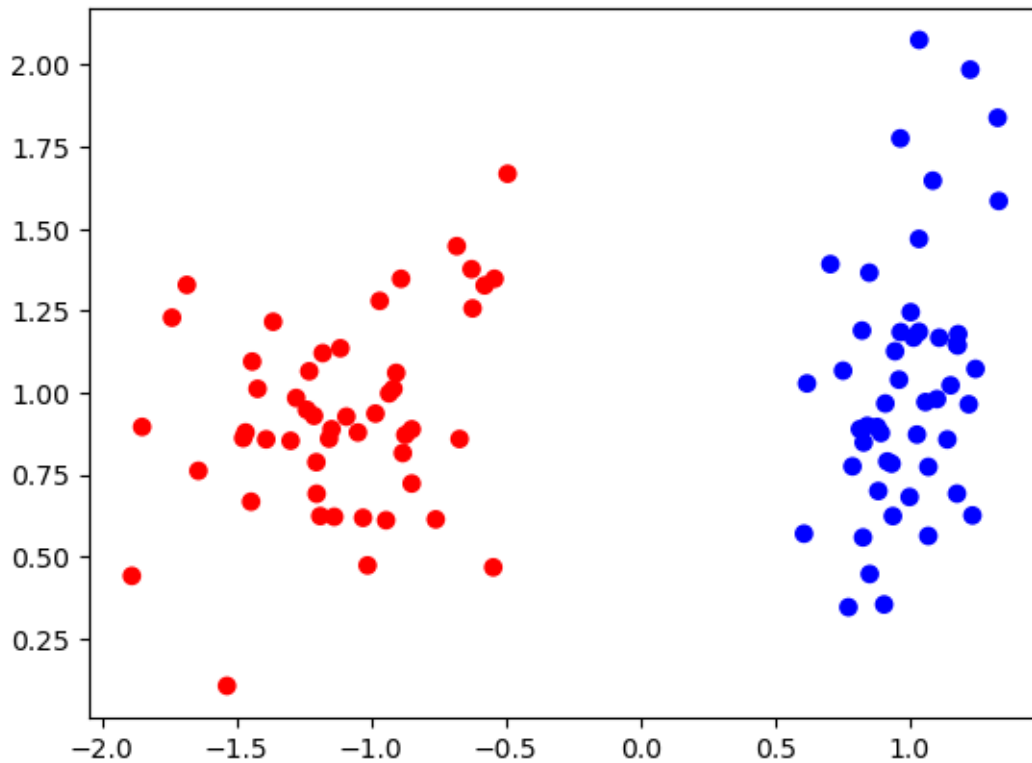
```
color = np.array(["r", "b", "y", "g", "k", "c", "orange", "tomato", "gray", "m"])
```

```
X, y = make_classification(n_features=2, n_redundant=0, n_informative=2,
                           random_state=1, n_clusters_per_class=1)
plt.scatter(X[:, 0], X[:, 1], c=color[y])
plt.show()
```



Esta es la separación original de los clusters

```
y_c = DBSCAN(eps=.5, min_samples=2).fit_predict(X)
plt.scatter(X[:, 0], X[:, 1], c=color[y_c])
plt.show()
```

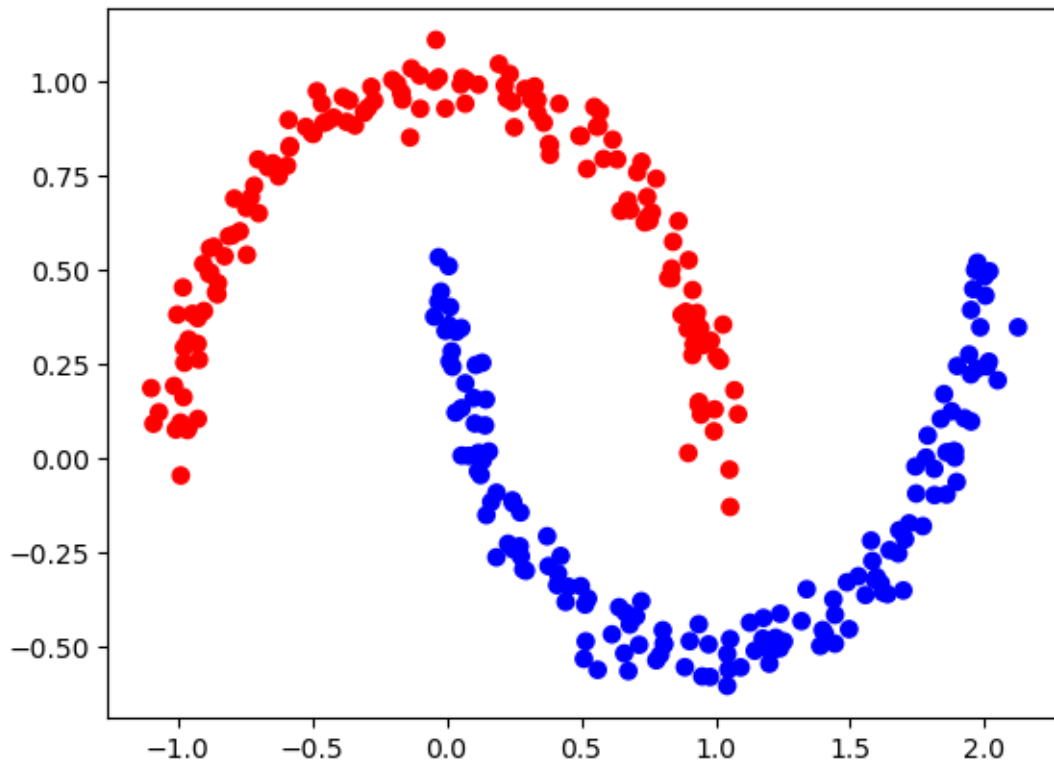


DBSCAN separa muy bien el conjunto de datos en este caso.

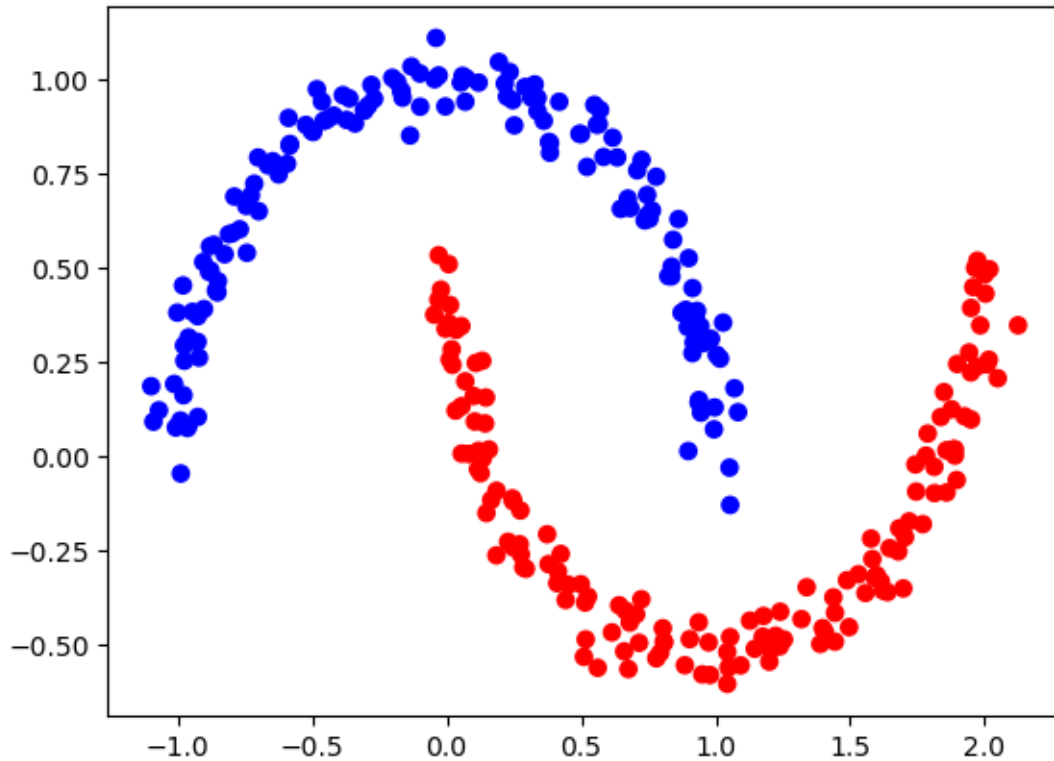
Medias lunas

Ahora veremos cómo se separan los datos con un conjunto de datos más complicado.

```
X, y = make_moons(n_samples=300, noise=0.05, random_state=0)
plt.scatter(X[:, 0], X[:, 1], c=color[y])
plt.show()
```



```
y_c = DBSCAN(eps=.2, min_samples=2).fit_predict(X)
plt.scatter(X[:, 0], X[:, 1], c=color[y_c])
plt.show()
```

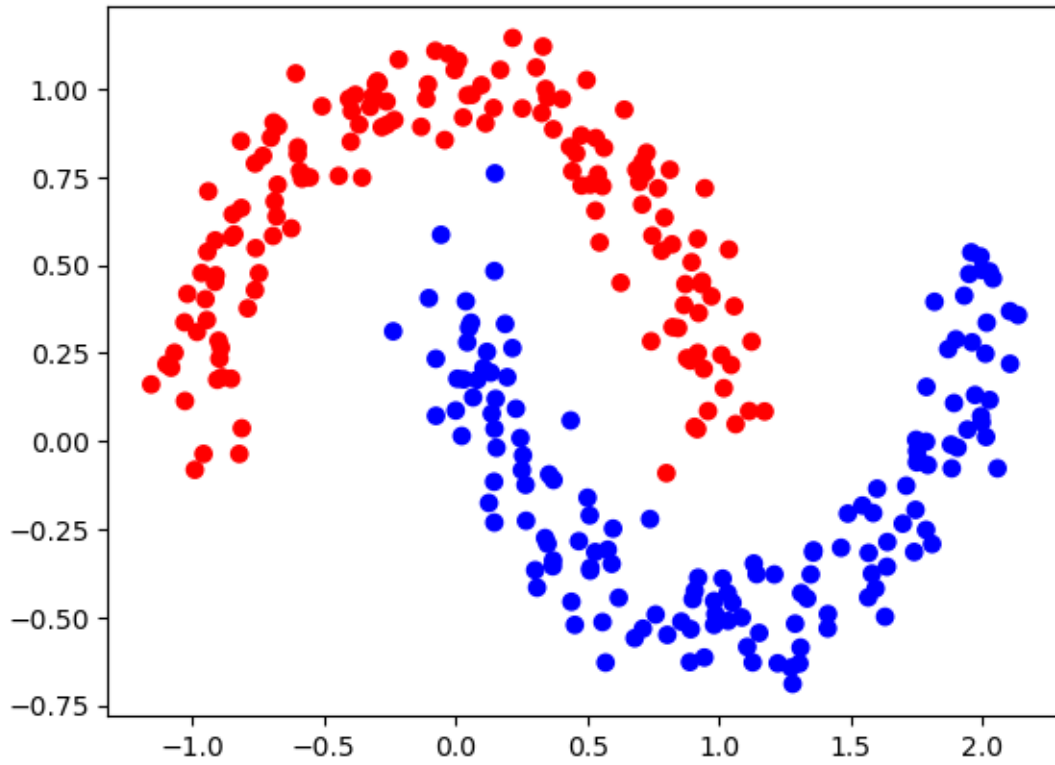


En el caso de las medias lunas, DBSCAN las separa perfectamente. Note que k-means no lo hubiera podido hacer.

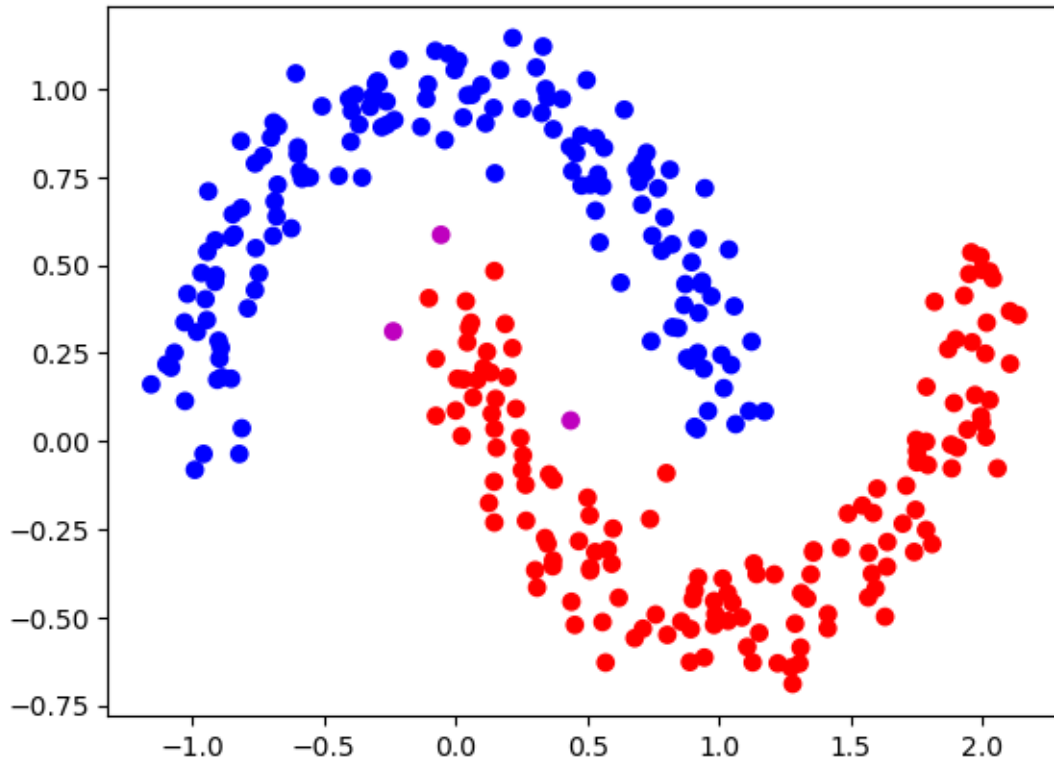
Muy juntas

Ahora vamos a poner las medias lunas muy juntas y ver el resultado de la partición.

```
X, y = make_moons(n_samples=300, noise=0.1, random_state=1)
plt.scatter(X[:, 0], X[:, 1], c=color[y])
plt.show()
```



```
y_c = DBSCAN(eps=.15, min_samples=2).fit_predict(X)
plt.scatter(X[:, 0], X[:, 1], c=color[y_c])
plt.show()
```

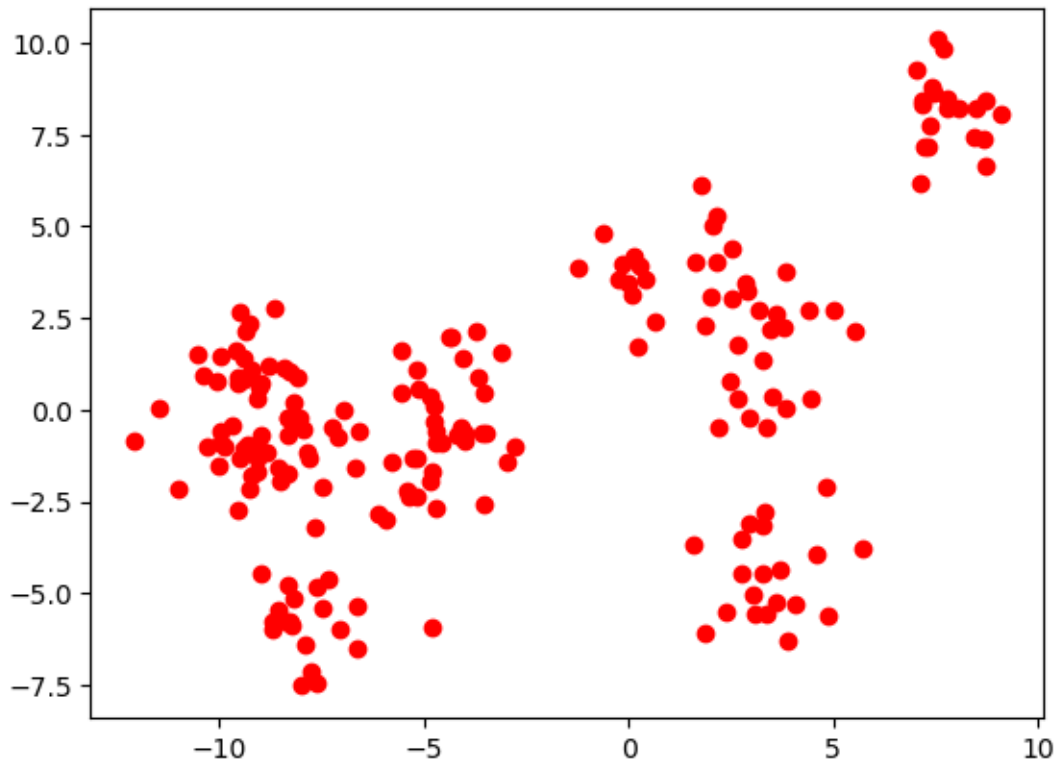


Se separan bien aún cuando las lunas están muy juntas. Los puntos morados son los que se consideran ruido.

Otras formas variando el radio

Aquí vamos a variar el radio y observar las diferentes formas en que se separan los datos.

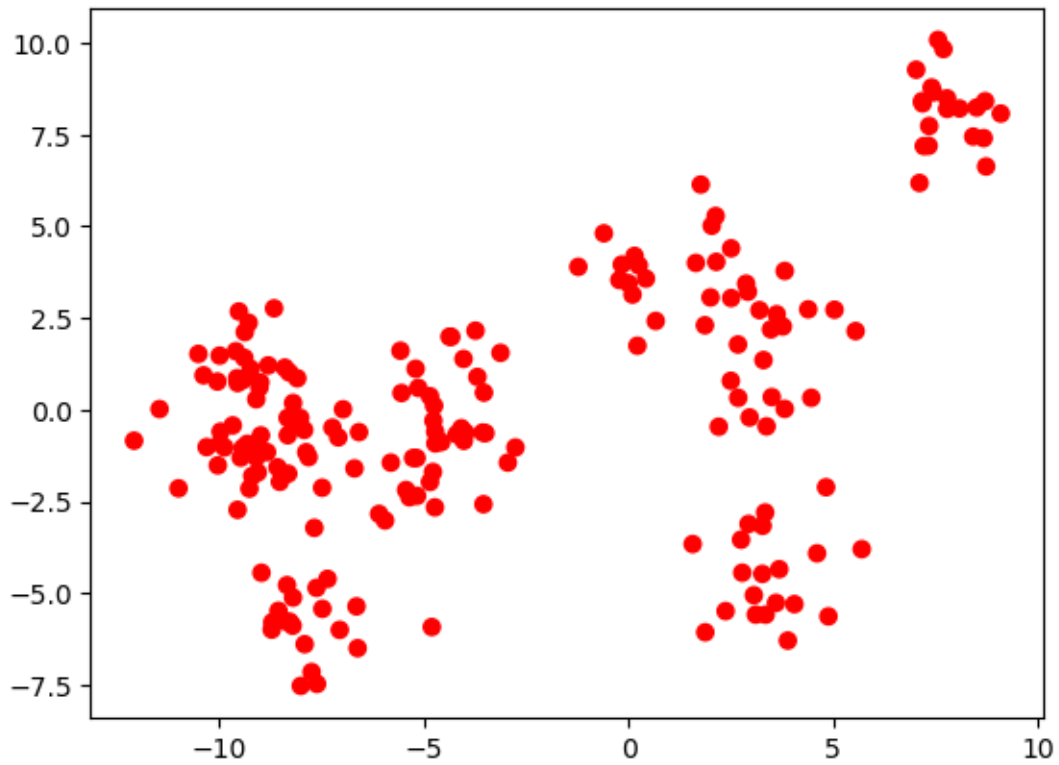
```
X, y = make_blobs(n_samples=200, centers=10, random_state=3)
plt.scatter(X[:, 0], X[:, 1], c=color[0])
plt.show()
```



Radio de 5

Primero con un radio de 5. Note que este radio es muy grande y el efecto es que todos los puntos se asignan al mismo cluster.

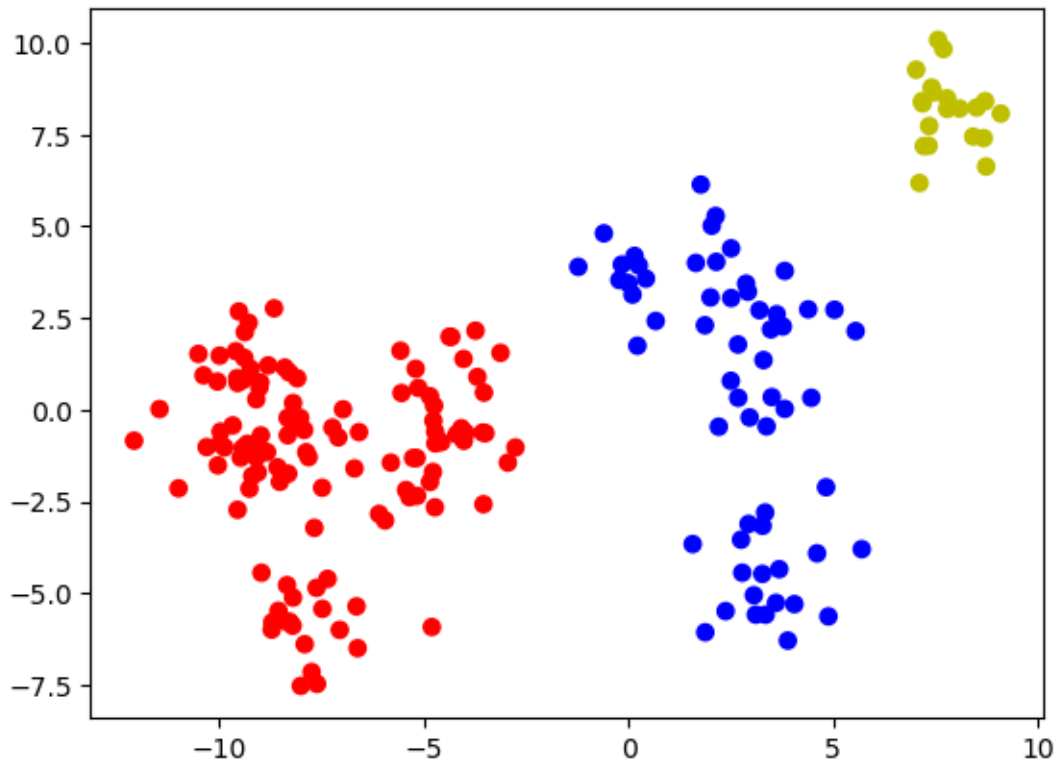
```
y_c = DBSCAN(eps=5, min_samples=5).fit_predict(X)
plt.scatter(X[:, 0], X[:, 1], c=color[y_c])
plt.show()
```

Radio = 3

Al ir disminuyendo el radio se van formando más clusters.

```
y_c = DBSCAN(eps=3, min_samples=5).fit_predict(X)
plt.scatter(X[:, 0], X[:, 1], c=color[y_c])
plt.show()
```

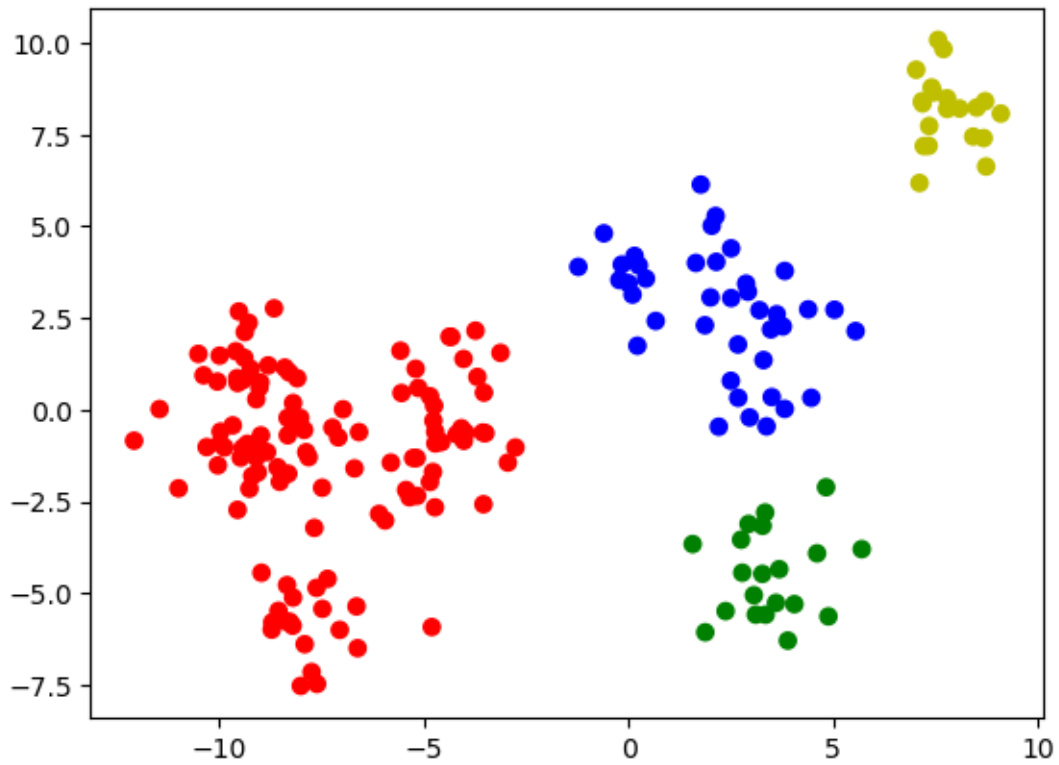


Con este radio, ya se crearon tres clusters.

Radio = 2

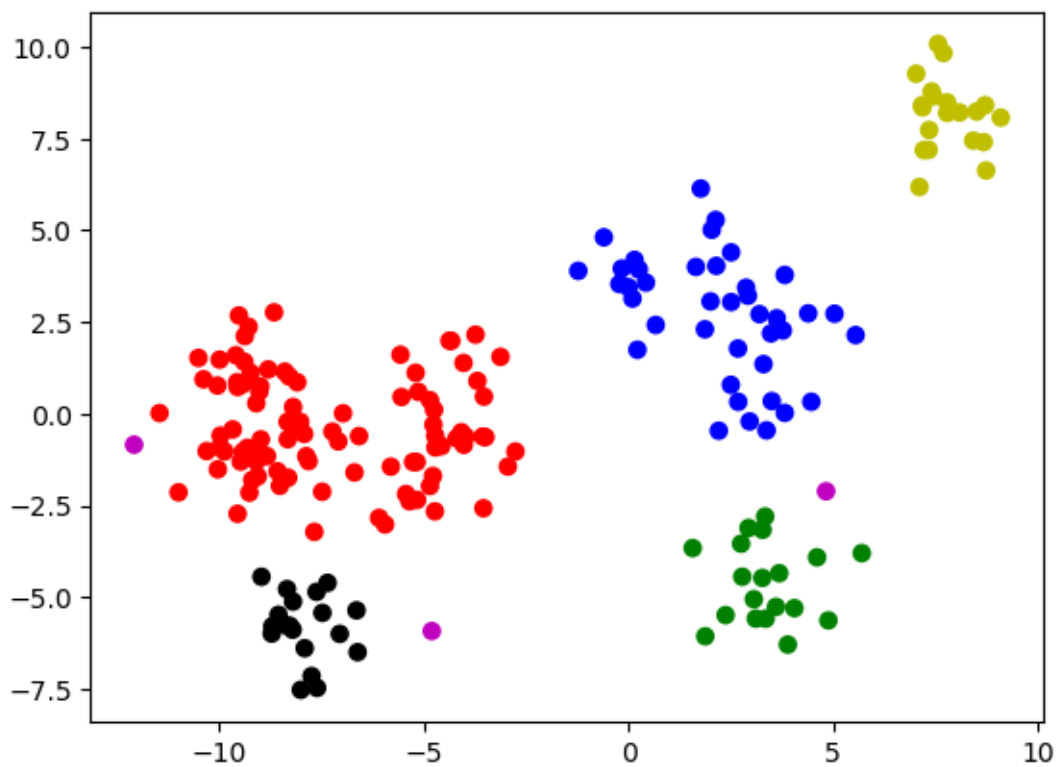
En este caso podemos ver que aparecen 4 clusters.

```
y_c = DBSCAN(eps=2, min_samples=5).fit_predict(X)
plt.scatter(X[:, 0], X[:, 1], c=color[y_c])
plt.show()
```



Radio = 1.5

```
y_c = DBSCAN(eps=1.5, min_samples=5).fit_predict(X)
plt.scatter(X[:, 0], X[:, 1], c=color[y_c])
plt.show()
```



Los magentas son los puntos marcados como ruido.

Actividad: Cambiar el conjunto de datos y los parámetros para comparar los resultados.